

EDR PREREQUISITES: COMPLETE CORE KNOWLEDGE STACK



INTRODUCTION

Endpoint Detection & Response (EDR) is not a malware finder but a high-fidelity telemetry system that amplifies analyst thinking, for better or worse, and it only works when foundational understanding exists. Before using EDR, an analyst must understand how Windows endpoints actually operate (processes, threads, execution chains, services, tasks, registry, file system), because detection depends on knowing what “normal” looks like. Modern attacks are process- and memory-centric, not file-centric, frequently abusing signed, trusted binaries and running in memory, which makes command lines, execution flow, API usage, and runtime behavior more important than file hashes. EDR observes behavior, not intent, and it builds on existing Windows security controls such as identities, tokens, permissions, LSASS, logging, and built-in protections rather than replacing them. High-value signals like persistence, credential access, identity abuse, and abnormal network behavior only make sense when analyzed in context, relative to endpoint role and baseline behavior. Alerts are summaries, not truth; telemetry and metadata are evidence, and investigations require time-based and cross-process correlation across an attacker’s lifecycle rather than alert-by-alert reactions. Response actions are not neutral and can destroy evidence or disrupt business if used emotionally instead of surgically. Ultimately, EDR is effective only when analysts think in behaviors, lifecycles, and hypotheses, validate context, document assumptions, and apply discipline—without this core knowledge stack, EDR produces noise and false confidence instead of security.

1. Endpoint Fundamentals

(You cannot monitor what you don't understand)

Before EDR, you must understand **how an endpoint actually works**.

Operating System Internals (Windows first)

You must know:

- **how an endpoint actually works.**
- **User mode vs Kernel mode**
- **Processes vs Threads**
- **Parent-child process relationships**
- **Services vs scheduled tasks**
- Registry (hives, keys, common persistence locations)
- File system structure (System32, ProgramData, AppData, Temp)

Key rule:

If you don't know **what normal looks like**, EDR alerts are meaningless noise.

Endpoint Fundamentals

(You cannot monitor what you don't understand)

1. Definition (Beginner-Level)

An **endpoint** is a device used by a person to do work, such as a laptop or desktop computer.

When we talk about *endpoint fundamentals*, we mean understanding **how the operating system (Windows)** actually runs programs, manages users, stores data, and performs background tasks.

Before using EDR (Endpoint Detection & Response), you must understand:

- What is running on the machine
- Why it is running
- Where it runs from
- How the operating system controls it

Without this understanding, security alerts have no context and cannot be evaluated correctly.

2. Purpose (Why This Exists)

This topic exists to solve **one core problem**:

Security tools generate data, but data is useless without understanding what "normal" looks like.

In real environments:

- EDR tools monitor processes, files, registry, and memory.
- These behaviors can be **legitimate or malicious**.
- The same action can be normal on one system and suspicious on another.

Endpoint fundamentals allow you to:

- Distinguish **normal OS behavior vs abuse**
- Reduce false positives

- Understand why alerts fire
- Avoid reacting blindly to noise

Key rule enforced here:

If you don't know what normal looks like, EDR alerts are meaningless noise.

3. Real-World Example (Concrete)

A Windows user opens Microsoft Word.

What actually happens:

- winword.exe starts as a **process**
- It runs in **user mode**
- It is launched by a **parent process** (usually explorer.exe)
- It reads files from C:\Users\<user>\Documents
- It may access registry keys related to user preferences

An EDR alert showing:

- winword.exe spawning powershell.exe
- From AppData\Temp
- With no user interaction

Is **not normal**, but you can only recognize this if you understand:

- Normal parent-child relationships
 - Normal file paths
 - Normal execution context
-

4. How It Works (Internal Logic – Step by Step)

4.1 User Mode vs Kernel Mode

- **User Mode:**
Where normal applications run (browsers, Office, scripts).
Limited permissions. Cannot directly access hardware.
- **Kernel Mode:**
Where the OS core runs. Full control over memory, CPU, drivers.

Security relevance:

- Malware trying to reach kernel mode is attempting **deep control**
 - Most user applications should never operate in kernel mode
-

4.2 Processes vs Threads

- A **process** is a running program (e.g., chrome.exe)
- A **thread** is a unit of execution inside a process

Security relevance:

- One process can contain many threads doing different tasks
 - Malicious code often injects threads into legitimate processes
-

4.3 Parent–Child Process Relationships

- Every process is usually started by another process
- Example:
 - explorer.exe → cmd.exe
 - cmd.exe → powershell.exe

Security relevance:

- Some parent–child chains are normal
- Others indicate abuse or automation

Understanding relationships helps detect:

- Script-based attacks
 - LOLBin abuse
 - Malicious execution chains
-

4.4 Services vs Scheduled Tasks

- **Services:**
 - Long-running background programs
 - Start at boot or system startup
- **Scheduled Tasks:**

- Run at specific times or triggers (logon, idle, etc.)

Security relevance:

- Both are common **persistence mechanisms**
 - Legitimate software and malware use the same mechanisms
-

4.5 Windows Registry

- Central configuration database
- Organized into **hives** and **keys**

Important concepts:

- Hives store large sections of settings
- Keys store specific values

Security relevance:

- Malware often uses registry for persistence
 - Certain locations are commonly abused (startup-related keys)
-

4.6 File System Structure

Key directories:

- **System32**: Core Windows binaries
- **ProgramData**: Shared application data
- **AppData**: User-specific application data
- **Temp**: Temporary files

Security relevance:

- Some paths are expected for execution
 - Others are suspicious depending on context
 - Malware often hides in user-writable directories
-

5. Simplified Explanation (Non-Technical)

Think of a computer like an office building:

- **Kernel mode** is the building's security room
- **User mode** is where employees work
- **Processes** are people doing jobs
- **Threads** are the tasks they perform
- **Parent-child processes** are who told whom to do what
- **Registry** is the rulebook
- **File system folders** are storage rooms

If you don't know what normal office activity looks like, you cannot spot a thief.

6. Practical Scenario (Realistic Use)

You receive an EDR alert:

- A process started from AppData\Temp
- It created a registry key for persistence
- It runs every user logon via a scheduled task

Using endpoint fundamentals, you evaluate:

- Is execution from Temp normal?
- Should this application create scheduled tasks?
- Does the parent process make sense?
- Is this behavior expected for this endpoint type?

Decision-making is based on **understanding**, not the alert itself.

7. Common Misunderstandings

1. **"EDR tells me what's bad"**
Incorrect. EDR shows behavior, not intent.
2. **"Signed binaries are safe"**
Incorrect. Legitimate binaries can be abused.
3. **"One alert equals one incident"**
Incorrect. Alerts are signals, not conclusions.
4. **"All PowerShell usage is malicious"**
Incorrect. Context defines legitimacy.

5. **“If it runs, it must be allowed”**

Incorrect. Many attacks run quietly within normal mechanisms.

8. Summary (Concise)

- Endpoints must be understood before monitoring them
 - Windows internals define what “normal” means
 - Processes, threads, and execution chains matter
 - Services, tasks, registry, and file paths are core persistence areas
 - EDR alerts without context are noise
 - Understanding normal behavior is mandatory for detection
-

2. Process, Execution & Memory Behavior (Critical)

EDR is **process- and memory-centric**, not file-centric.

You must understand:

- How a process is created
- Command-line arguments and switches
- Process injection (high-level logic, not exploit dev)
- DLL loading vs reflective / in-memory loading
- File-based vs fileless execution
- Signed vs unsigned binaries

Example you must reason **without tools**:

Why powershell.exe spawned by winword.exe with encoded commands is suspicious

If you cannot reason about execution flow and memory behavior, EDR will overwhelm you.

Process, Execution & Memory Behavior (Critical)

EDR is process- and memory-centric, not file-centric.

1. Definition (Beginner-Level)

Process, execution, and memory behavior describe **how programs start, how they run, and how they use memory inside an operating system.**

This topic focuses on:

- How a program becomes a running process
- How instructions are passed to it
- How it loads code into memory
- Whether it relies on files or runs purely in memory

EDR does not primarily look at files.

It watches **process behavior and memory activity.**

2. Purpose (Why This Exists)

This topic exists to solve a critical operational problem:

Modern attacks often do not rely on obvious malicious files.

In real environments:

- Malware may never touch disk
- Legitimate tools may be abused
- Memory activity reveals intent more reliably than files

Understanding process and memory behavior allows you to:

- Detect fileless attacks
- Identify abnormal execution chains
- Reason about intent without signatures
- Avoid alert fatigue from benign binaries

If you cannot reason about execution flow and memory behavior, EDR data becomes overwhelming noise.

3. Real-World Example (Concrete)

A user opens a Word document.

Observed behavior:

- winword.exe launches
- Shortly after, powershell.exe starts
- PowerShell runs with encoded command-line arguments
- No script file exists on disk

There is no malware file to scan.

Detection depends entirely on:

- Process relationship
 - Command-line content
 - Execution context
 - Memory behavior
-

4. How It Works (Internal Logic – Step by Step)

4.1 How a Process Is Created

- A user or system action requests execution
- The OS loads the executable into memory
- Memory space is allocated
- Threads are created
- The process begins execution

Security relevance:

- Process creation always has **context**
 - Who started the process matters as much as what started
-

4.2 Command-Line Arguments and Switches

- Processes often receive instructions at launch
- These arguments control behavior

Security relevance:

- Malicious intent is frequently hidden in arguments
 - Encoded or obfuscated arguments reduce visibility
 - Same binary, different arguments → different intent
-

4.3 Process Injection (High-Level)

- One process inserts code into another process's memory
- The injected code runs under the victim process identity

Security relevance:

- No new process may appear
- Trusted processes can be hijacked
- Memory telemetry becomes critical

(No exploit techniques are required to understand the concept.)

4.4 DLL Loading vs Reflective / In-Memory Loading

- **Normal DLL loading:**
 - DLL is loaded from disk
 - Registered with the OS loader
- **Reflective / in-memory loading:**
 - DLL never touches disk
 - Loaded directly into memory

Security relevance:

- Disk-based detection may fail
 - Memory-only activity is a strong signal
 - EDR visibility into memory is essential
-

4.5 File-Based vs Fileless Execution

- **File-based execution:**
 - Executable exists on disk
 - Traditional malware model
- **Fileless execution:**
 - Code runs from memory
 - Scripts, injection, or reflective loading

Security relevance:

- Fileless attacks evade traditional antivirus
 - EDR focuses on behavior, not files
-

4.6 Signed vs Unsigned Binaries

- **Signed binaries:**
 - Verified publisher
 - Trusted by default
- **Unsigned binaries:**
 - No publisher validation

Security relevance:

- Signed does not mean safe
 - Many attacks abuse signed system tools
 - Context defines legitimacy, not signature
-

5. Simplified Explanation (Non-Technical)

Think of a process like a worker:

- The **process** is the worker
- The **command line** is the instruction note
- **Memory** is the workspace
- **Injection** is someone secretly taking control of the worker
- **Fileless execution** is working without leaving paperwork

Security is about **how the work happens**, not just who the worker is.

6. Practical Scenario (Reasoning Without Tools)

Scenario:

- powershell.exe is launched
- Parent process is winword.exe
- Command line contains encoded instructions

Reasoning:

- Word normally edits documents, not runs scripts
- PowerShell is capable of executing arbitrary commands
- Encoded arguments hide behavior
- Execution chain suggests automation, not user intent

Conclusion:

- Suspicion comes from **execution flow + memory usage**
 - No malware file is required to reach this conclusion
-

7. Common Misunderstandings

1. **"No malicious file means no attack"**
Incorrect. Many attacks are fileless.
 2. **"Signed binaries are always safe"**
Incorrect. Abuse is common.
 3. **"Only new processes matter"**
Incorrect. Injection hides inside existing processes.
 4. **"PowerShell is bad by default"**
Incorrect. Context and arguments matter.
 5. **"AV and EDR work the same way"**
Incorrect. EDR focuses on execution and memory.
-

8. Summary (Concise)

- EDR focuses on processes and memory, not files
- Process creation context is critical

- Command-line arguments reveal intent
- Injection and in-memory loading bypass disk detection
- Fileless execution is common in modern attacks
- Execution flow understanding is mandatory for effective EDR analysis

3. Windows API & Runtime Awareness

You don't need to code, but you must understand **what EDR actually observes**.

You must know:

- What Windows APIs are (conceptually)
- Why malware uses APIs instead of visible "commands"
- User-mode API calls vs direct syscalls (evasion idea)
- Why EDR hooks runtime behavior, not attacker intent

Without this, API-sequence or "behavioral" alerts will look like nonsense.

Windows API & Runtime Awareness

You don't need to code, but you must understand what EDR actually observes.

1. Definition (Beginner-Level)

Windows APIs are standardized ways for programs to ask the operating system to perform actions on their behalf.

Examples of actions include:

- Creating a process

- Reading or writing memory
- Accessing files or registry
- Communicating over the network

Programs do not directly control the operating system.

They **request actions through APIs**, and the OS decides how those actions happen.

EDR observes **these runtime interactions**, not the attacker's intentions.

2. Purpose (Why This Exists)

This topic exists to solve a core visibility problem:

Attackers do not "run commands" the way humans think — they invoke operating system functionality.

In real systems:

- Malware rarely types visible commands
- It calls APIs to achieve goals quietly
- Different tools can produce the same underlying API behavior

Understanding Windows APIs allows you to:

- Interpret behavioral alerts correctly
- Understand why EDR flags certain sequences
- Avoid assuming alerts are random or magical
- Recognize evasion techniques conceptually

Without this awareness, API-sequence or behavioral alerts appear meaningless.

3. Real-World Example (Concrete)

A malicious program wants to steal credentials.

What you might expect:

- A visible "dump credentials" command

What actually happens:

- APIs are called to access process memory
- APIs are used to read protected areas
- APIs write extracted data elsewhere

EDR does not see a "credential theft command."
It sees a **sequence of sensitive API calls at runtime**.

4. How It Works (Internal Logic – Step by Step)

4.1 What Windows APIs Are (Conceptually)

- APIs are predefined entry points into OS functionality
- They abstract complex system behavior
- All normal applications rely on them

Security relevance:

- Malware and legitimate software use the same APIs
 - Behavior, not presence, defines risk
-

4.2 Why Malware Uses APIs Instead of Visible "Commands"

- Commands are high-level human constructs
- APIs are low-level system interfaces
- APIs provide:
 - Precision
 - Stealth
 - Flexibility

Security relevance:

- Malware blends in with normal application behavior
 - No obvious command history exists
-

4.3 User-Mode API Calls vs Direct Syscalls (Evasion Concept)

- **User-mode API calls:**
 - Go through documented OS layers

- Easier for EDR to observe
- **Direct syscalls:**
 - Bypass some user-mode layers
 - Used to reduce visibility

Security relevance:

- Some malware attempts to evade monitoring
 - EDR looks for abnormal patterns, not single calls
 - You do not need exploit knowledge to understand the evasion idea
-

4.4 Why EDR Hooks Runtime Behavior, Not Attacker Intent

- Intent is not observable
- Runtime behavior is observable
- APIs and syscalls leave measurable traces

Security relevance:

- EDR instruments:
 - Process behavior
 - Memory access
 - API usage patterns
 - Alerts are based on **what happened**, not why it happened
-

5. Simplified Explanation (Non-Technical)

Think of Windows like a service desk:

- APIs are request forms
- Programs submit forms to get things done
- Malware fills out the same forms as normal programs

Security watches:

- Which forms were submitted
- In what order
- How frequently
- By which programs

Not the person's intent — the actions taken.

6. Practical Scenario (Realistic Use)

You see an EDR alert:

- A process performs a sequence of memory-access APIs
- Followed by process creation APIs
- Followed by network communication APIs

Even without knowing the exact APIs:

- The **sequence** indicates staging and execution
- The behavior is suspicious regardless of tool used

Your analysis is based on **runtime patterns**, not strings or commands.

7. Common Misunderstandings

1. **"If I don't see a command, nothing happened"**
Incorrect. APIs perform the real work.
 2. **"Only malware uses suspicious APIs"**
Incorrect. Context and sequence matter.
 3. **"EDR understands attacker goals"**
Incorrect. EDR only sees observable behavior.
 4. **"Direct syscalls mean invisibility"**
Incorrect. Patterns still exist.
 5. **"API alerts require coding knowledge"**
Incorrect. Conceptual understanding is sufficient.
-

8. Summary (Concise)

- Windows APIs are how programs interact with the OS
- Malware relies on APIs, not visible commands
- EDR monitors runtime behavior and API patterns
- User-mode vs syscall differences explain evasion attempts
- Intent is invisible; behavior is measurable

- API-sequence alerts require conceptual OS understanding
-

4. Windows Security Mechanisms (EDR builds on these)

You must know what already exists **before** EDR adds visibility.

Core concepts:

- User accounts, privileges, tokens, UAC
- NTFS permissions
- Credential storage basics (LSASS role)
- Event logs (Security, System, Application)
- Built-in protections (Defender, ASR rules, AMSI)

EDR does not replace Windows security — it **observes and augments** it.

Windows Security Mechanisms (EDR Builds on These)

EDR does not replace Windows security — it observes and augments it.

1. Definition (Beginner-Level)

Windows security mechanisms are the protections already built into the Windows operating system to control access, protect credentials, log activity, and block malicious behavior.

These mechanisms exist **before** EDR is installed.

EDR does not create security from scratch — it **monitors, correlates, and adds visibility** on top of what Windows already enforces.

Understanding these fundamentals is required to interpret EDR alerts correctly.

2. Purpose (Why This Exists)

This topic exists to address a common analytical failure:

Analysts treat EDR as the security system, instead of understanding it as an observer of Windows security controls.

In real environments:

- Windows enforces permissions and protections
- EDR watches how those protections are used, bypassed, or abused
- Alerts are meaningless if you don't understand the underlying control

Knowing Windows security mechanisms allows you to:

- Understand why an alert exists
 - Identify privilege abuse
 - Recognize credential access attempts
 - Interpret security logs correctly
 - Avoid assuming EDR "missed" something that Windows already blocked
-

3. Real-World Example (Concrete)

An alert shows:

- A process attempting to access credential-related memory
- Access is denied
- A security event is logged

Without Windows security knowledge:

- You may think the attack failed and ignore it

With Windows security knowledge:

- You recognize this as **attempted credential access**
- You understand which protection blocked it
- You correlate EDR telemetry with Windows event logs

- You treat it as malicious intent, not a harmless failure
-

4. How It Works (Internal Logic – Step by Step)

4.1 User Accounts, Privileges, Tokens, and UAC

- Every action runs under a **user account**
- Each account has assigned **privileges**
- When a user logs in, Windows issues a **security token**
- **UAC** controls privilege elevation

Security relevance:

- Malware inherits the user's token
 - Privilege escalation attempts target token elevation
 - EDR alerts often reflect abuse of privileges, not just execution
-

4.2 NTFS Permissions

- NTFS controls:
 - Who can read files
 - Who can write files
 - Who can execute files
- Permissions apply to files and directories

Security relevance:

- Malware often fails or succeeds based on NTFS permissions
 - Unexpected access attempts are meaningful signals
 - EDR correlates permission failures with process behavior
-

4.3 Credential Storage Basics (LSASS Role)

- Windows stores authentication material in memory
- The **Local Security Authority Subsystem Service (LSASS)** manages credentials

Security relevance:

- Credential theft targets LSASS
- Direct access attempts are highly sensitive
- EDR monitors processes interacting with LSASS-related memory

You do not need exploitation knowledge — only the role.

4.4 Event Logs (Security, System, Application)

- Windows logs key activity into:
 - **Security** logs (authentication, access, privilege use)
 - **System** logs (OS-level events)
 - **Application** logs (application behavior)

Security relevance:

- These logs are authoritative
 - EDR enriches and correlates them
 - Missing log context leads to incorrect conclusions
-

4.5 Built-in Protections (Defender, ASR, AMSI)

- Windows includes:
 - Antivirus protection
 - Attack Surface Reduction rules
 - Script inspection through AMSI

Security relevance:

- Many attacks are blocked before execution
 - EDR observes:
 - Block events
 - Scan results
 - Script inspection outcomes
 - An alert may indicate **attempt**, not success
-

5. Simplified Explanation (Non-Technical)

Think of Windows security like a building:

- User accounts are ID badges
- Permissions are locked doors
- LSASS is the key vault
- Event logs are security cameras
- Built-in protections are guards

EDR is the **control room** that watches all of this happen — it does not replace the locks or guards.

6. Practical Scenario (Realistic Use)

You investigate an EDR alert:

- A script was blocked
- AMSI inspection flagged suspicious content
- A security log shows denied access
- No malicious process is running

Correct interpretation:

- Windows protections worked
- EDR recorded and correlated the attempt
- The absence of compromise does not equal benign behavior

Response decisions depend on understanding **Windows controls**, not just EDR severity.

7. Common Misunderstandings

1. **“EDR replaces Windows security”**
Incorrect. EDR builds on it.
2. **“Blocked means safe”**
Incorrect. Blocked attempts still indicate threat activity.
3. **“Credentials are files on disk”**
Incorrect. They are primarily memory-resident.
4. **“Logs are secondary to EDR alerts”**
Incorrect. Logs are primary evidence.
5. **“If Defender alerts, EDR is redundant”**
Incorrect. EDR provides correlation and context.

8. Summary (Concise)

- Windows security mechanisms exist before EDR
- Accounts, tokens, and privileges define execution context
- NTFS permissions control access
- LSASS manages credentials
- Event logs provide authoritative records
- Built-in protections block many attacks
- EDR observes, correlates, and augments — it does not replace

5. Persistence Mechanisms (Explicit and Detailed)

Persistence is one of the **highest-value EDR detections**.

You must explicitly know:

- Registry Run / RunOnce keys
- Scheduled tasks abuse
- Service creation and modification
- Startup folders
- WMI event subscriptions

If you can't instantly recognize persistence attempts, you will rely blindly on verdicts.

Persistence Mechanisms (Explicit and Detailed)

Persistence is one of the highest-value EDR detections.

1. Definition (Beginner-Level)

Persistence is any technique used to **make malicious or unauthorized code run again after a reboot, logoff, or system restart**.

An attack is not complete if it runs once.

Persistence ensures:

- Re-execution
- Long-term access
- Continued control

EDR places high value on persistence because **legitimate software and attackers use the same mechanisms**, and misuse stands out clearly when you understand normal behavior.

2. Purpose (Why This Exists)

This topic exists to solve a critical detection failure:

Analysts often see persistence alerts but do not understand what is actually being persisted.

In real environments:

- Initial execution may be missed
- Persistence remains visible for long periods
- Persistence strongly indicates **intent**, not accident

Understanding persistence allows you to:

- Detect attacks even after the initial payload is gone
- Identify footholds

- Prioritize alerts correctly
- Avoid blind trust in EDR verdicts

If you cannot instantly recognize persistence attempts, you will rely blindly on verdicts instead of reasoning.

3. Real-World Example (Concrete)

A laptop is rebooted.

Observed behavior:

- A PowerShell script runs automatically
- No user launches it
- No installer was recently run

Investigation shows:

- A registry key launches the script at logon

This is not normal user behavior.

The system was deliberately configured to **re-execute code**.

4. How It Works (Internal Logic – Step by Step)

4.1 Registry Run / RunOnce Keys

- Windows checks specific registry locations at user logon or system startup
- Entries define programs to execute automatically

Security relevance:

- Very common persistence method
- Easy to abuse
- Often used by both malware and commodity tools
- Execution happens silently

Key idea:

- Anything executing from these keys must be justified

4.2 Scheduled Tasks Abuse

- Scheduled tasks allow programs to run:
 - At logon
 - On a schedule
 - On system events
- Tasks can run hidden and with elevated privileges

Security relevance:

- Flexible and powerful
- Tasks can survive reboots
- Tasks can impersonate legitimate maintenance jobs

Persistence signal:

- Unexpected tasks
 - Tasks running scripts or binaries from user-writable paths
-

4.3 Service Creation and Modification

- Windows services:
 - Start automatically
 - Run in the background
 - Often run with high privileges

Security relevance:

- Creating a service establishes strong persistence
- Modifying an existing service can hijack trusted behavior
- Service persistence is high-impact

EDR focus:

- New services
 - Changes to service binaries or startup types
-

4.4 Startup Folders

- Windows supports startup folders for automatic execution
- Files placed here run at user logon

Security relevance:

- Simple and effective
- Often overlooked
- User-specific or system-wide

Persistence signal:

- Executables or scripts appearing without user awareness
 - Unexpected file types
-

4.5 WMI Event Subscriptions

- WMI allows automation based on system events
- Code can be triggered by conditions, not time

Security relevance:

- Less visible than other methods
- No obvious startup entry
- Survives reboot

Persistence signal:

- WMI activity without administrative justification
 - Event-triggered execution chains
-

5. Simplified Explanation (Non-Technical)

Persistence is like setting an alarm clock:

- It runs something automatically
- Without asking again
- Every time conditions are met

Security cares because:

- Normal users rarely set hidden alarms

- Attackers always do
-

6. Practical Scenario (Realistic Use)

You receive an EDR alert:

- A new scheduled task was created
- Task runs a script from AppData
- Triggered at user logon

Reasoning:

- AppData is user-writable
- Logon trigger implies persistence
- Script execution implies automation

Conclusion:

- This is not accidental
 - Persistence was intentionally established
 - High-priority investigation is required
-

7. Common Misunderstandings

1. **“Persistence means malware is active right now”**
Incorrect. It means re-execution is configured.
 2. **“Startup entries are always benign”**
Incorrect. Context defines legitimacy.
 3. **“Only advanced malware uses persistence”**
Incorrect. Persistence is universal.
 4. **“One persistence method equals one alert”**
Incorrect. Multiple mechanisms are often layered.
 5. **“If EDR flags it, I don’t need to understand it”**
Incorrect. Understanding determines response quality.
-

8. Summary (Concise)

- Persistence ensures repeated execution
 - Registry Run keys are common and silent
 - Scheduled tasks provide flexible automation
 - Services offer high-privilege persistence
 - Startup folders are simple but effective
 - WMI subscriptions are stealthy
 - Persistence detection is high-confidence signal
 - Analysts must recognize persistence instantly
-

6. LOLBins & Trusted Binary Abuse

Understanding the concept of LOLBins is not enough.
You must recognize them **by name and behavior**.

You must know common examples:

- powershell.exe
- rundll32.exe
- mshta.exe
- certutil.exe
- wmic.exe
- regsvr32.exe

Why this matters:

- EDR alerts often involve **legitimate signed binaries**
- No malware file exists — only abuse

If a binary name doesn't immediately trigger context in your head, you are not ready.

LOLBins & Trusted Binary Abuse

Understanding the concept is not enough — recognition by name and behavior is mandatory.

1. Definition (Beginner-Level)

LOLBins (Living-Off-the-Land Binaries) are **legitimate, built-in Windows executables** that attackers abuse to perform malicious actions.

These binaries:

- Are already present on the system
- Are digitally signed and trusted
- Perform powerful system functions

No new malware file is required.

The attack exists purely as **abuse of normal tools**.

2. Purpose (Why This Exists)

This topic exists to solve a fundamental detection failure:

Analysts look for “malware files” while attackers use trusted binaries.

In real environments:

- EDR alerts frequently reference signed Windows executables
- There may be no dropped payload
- Traditional “bad file” logic fails completely

Understanding LOLBins allows you to:

- Interpret alerts involving trusted binaries
- Detect abuse through context and behavior
- Avoid dismissing high-risk activity as “normal Windows”

If a binary name does not immediately trigger context, you are not ready to analyze EDR alerts.

3. Real-World Example (Concrete)

An EDR alert shows:

- rundll32.exe executing
- Loading a DLL from a user-writable directory
- No installer activity occurred

There is no malware executable.

The detection relies entirely on:

- Binary identity
 - Execution context
 - Behavior mismatch
-

4. How It Works (Internal Logic – Step by Step)

4.1 Why Trusted Binaries Are Abused

- They are already allowed by policy
- They bypass application allow-listing
- They blend into normal system activity

Security relevance:

- Trust is based on **signature**, not intent
 - Attackers exploit this trust boundary
-

4.2 Common LOLBins You Must Recognize

powershell.exe

- Legitimate purpose: automation and administration
- Abuse pattern:
 - Encoded commands

- Script execution without files
- Launched by non-interactive parents

Security signal:

- Context, arguments, and parent process matter more than existence
-

rundll32.exe

- Legitimate purpose: execute DLL functions
- Abuse pattern:
 - DLLs from unexpected locations
 - Used as a generic loader

Security signal:

- Rarely used directly by users
 - Highly sensitive to execution context
-

mshta.exe

- Legitimate purpose: execute HTML applications
- Abuse pattern:
 - Executes remote or embedded scripts
 - Often used for initial execution

Security signal:

- Almost never required in modern environments
-

certutil.exe

- Legitimate purpose: certificate management
- Abuse pattern:
 - Downloading files
 - Encoding/decoding payloads

Security signal:

- Network usage is highly suspicious outside admin tasks

wmic.exe

- Legitimate purpose: system management queries
- Abuse pattern:
 - Remote command execution
 - Reconnaissance

Security signal:

- Deprecated but still abused
- Rare in normal user workflows

regsvr32.exe

- Legitimate purpose: register COM components
- Abuse pattern:
 - Scriptlet execution
 - No DLL registration intent

Security signal:

- Registration without software installation context is suspicious

4.3 Why EDR Alerts Often Involve These Binaries

- They are trusted and signed
- They perform sensitive operations
- Abuse produces **behavioral anomalies**, not malware signatures

EDR detects:

- Unusual parents
- Suspicious command lines
- Abnormal execution paths
- Unexpected network or memory activity

5. Simplified Explanation (Non-Technical)

Think of LOLBins as company tools:

- Everyone trusts them
- They have powerful capabilities
- Attackers misuse them instead of bringing weapons

Security looks at **how the tool is used**, not the tool itself.

6. Practical Scenario (Realistic Use)

Alert details:

- powershell.exe started by winword.exe
- Command line contains encoded content
- No script file exists

Reasoning:

- Word does not normally automate PowerShell
- Encoding hides intent
- Execution is non-interactive

Conclusion:

- This is trusted binary abuse
 - No malware file is required
 - High-confidence malicious activity
-

7. Common Misunderstandings

1. **“Signed means safe”**
Incorrect. Signed means trusted by the OS, not benign.
2. **“PowerShell alerts are always false positives”**
Incorrect. Context defines risk.
3. **“If no file is dropped, nothing happened”**
Incorrect. Abuse is the attack.
4. **“LOLBins are advanced-only techniques”**
Incorrect. They are widely used at all skill levels.
5. **“EDR should block these binaries entirely”**
Incorrect. They are required for system operation.

8. Summary (Concise)

- LOLBins are legitimate Windows binaries
 - Attackers abuse them instead of dropping malware
 - Common examples must be recognized instantly
 - Context, arguments, and parent process define intent
 - EDR alerts frequently involve trusted binaries
 - No malware file is required for compromise
 - Recognition speed is critical for effective analysis
-

7. Networking Basics (Endpoint Perspective)

EDR sees **endpoint network behavior**, not packet captures.

You must understand:

- TCP vs UDP (behavioral implications)
- DNS resolution flow
- Outbound vs inbound connections
- Localhost vs external traffic
- Common protocols and ports (HTTP/S, SMB, RDP)

Example:

Why outbound SMB or LDAP from a workstation is usually abnormal

Networking Basics (Endpoint Perspective)

EDR sees endpoint network behavior, not packet captures.

1. Definition (Beginner-Level)

Networking from an endpoint perspective means understanding **how a workstation or server initiates and receives network connections**, as observed by EDR.

EDR does **not** see raw packets.
It observes:

- Which process initiated a connection
- Where the connection went
- Which protocol and port were used
- Whether the traffic stayed local or left the machine

The focus is **behavior**, not protocol internals.

2. Purpose (Why This Exists)

This topic exists to correct a common analytical error:

Analysts think like network engineers while EDR thinks like the endpoint.

In real environments:

- Most attacks involve network communication
- The *process* initiating traffic matters more than packet contents
- Abnormal connections are often obvious **only when endpoint context is applied**

Understanding endpoint networking allows you to:

- Identify command-and-control behavior
- Detect lateral movement attempts
- Reduce false positives
- Reason about intent without deep packet inspection

Without this, network-related EDR alerts are misinterpreted or ignored.

3. Real-World Example (Concrete)

An EDR alert shows:

- powershell.exe initiates a network connection
- Destination is an internal server
- Protocol is SMB

No packet data is available.

The alert is entirely based on **process + destination + protocol**.

Correct interpretation requires endpoint networking knowledge, not PCAP analysis.

4. How It Works (Internal Logic – Step by Step)

4.1 TCP vs UDP (Behavioral Implications)

- **TCP:**
 - Connection-oriented
 - Reliable
 - Used by most applications
- **UDP:**
 - Connectionless
 - Faster
 - Less common for application logic on endpoints

Security relevance:

- Most malicious tooling uses TCP for reliability
 - Unexpected UDP usage by user applications is notable
 - EDR tracks connection attempts and duration, not packets
-

4.2 DNS Resolution Flow

- Endpoints resolve domain names before connecting
- DNS queries precede most outbound connections

Security relevance:

- DNS activity often reveals intent early
- Suspicious domains may appear before payload delivery
- EDR correlates DNS requests with subsequent connections

Understanding DNS flow helps explain:

- Why alerts appear before network connections succeed
 - Why blocked connections still generate telemetry
-

4.3 Outbound vs Inbound Connections

- **Outbound:**
 - Initiated by the endpoint
 - Common for workstations
- **Inbound:**
 - Services listening for connections
 - Rare on user endpoints

Security relevance:

- Unexpected inbound listeners indicate compromise
- Outbound connections to unusual services indicate misuse

EDR emphasizes **who initiated** the connection.

4.4 Localhost vs External Traffic

- **Localhost (127.0.0.1):**
 - Traffic stays on the machine
- **External traffic:**
 - Leaves the endpoint

Security relevance:

- Localhost traffic may indicate:
 - Tunneling
 - Inter-process communication
- External traffic may indicate:
 - Data exfiltration
 - Command-and-control

EDR distinguishes local vs external destinations clearly.

4.5 Common Protocols and Ports

- **HTTP/HTTPS:**
 - Common outbound traffic
 - Often abused due to permissiveness
- **SMB:**
 - File sharing
 - Normally server-to-server or client-to-server
- **RDP:**
 - Remote access
 - Rarely initiated by workstations outward

Security relevance:

- Protocol legitimacy depends on **endpoint role**
 - Same port can be benign or suspicious depending on context
-

5. Simplified Explanation (Non-Technical)

Think of the endpoint as a person making calls:

- TCP is a phone call
- UDP is a text message
- DNS is looking up a phone number
- Outbound calls are normal
- Unexpected incoming calls are suspicious

Security cares **who called whom and why**, not the conversation content.

6. Practical Scenario (Reasoning Example)

Scenario:

- A workstation initiates an outbound SMB connection
- Destination is another workstation
- Initiated by a script interpreter

Reasoning:

- Workstations do not normally offer SMB services
- SMB is commonly used for lateral movement
- Script-initiated network activity is high risk

Conclusion:

- Outbound SMB from a workstation is usually abnormal
- Strong indicator of lateral movement or reconnaissance

Same logic applies to outbound LDAP from non-server endpoints.

7. Common Misunderstandings

1. **"No packet capture means no visibility"**
Incorrect. Process-level telemetry is sufficient.
 2. **"All HTTP traffic is benign"**
Incorrect. Context defines intent.
 3. **"DNS alerts are low value"**
Incorrect. DNS often precedes compromise.
 4. **"Inbound traffic is always bad"**
Incorrect. Role defines normal behavior.
 5. **"Network alerts belong to the network team"**
Incorrect. Endpoint context is critical.
-

8. Summary (Concise)

- EDR observes endpoint network behavior, not packets
- TCP vs UDP differences affect detection patterns
- DNS resolution reveals early intent

- Outbound vs inbound initiation matters
- Localhost vs external traffic changes interpretation
- Protocol legitimacy depends on endpoint role
- Outbound SMB or LDAP from workstations is usually abnormal

8. Malware Fundamentals (Behavior, Not Labels)

Before EDR terms, you must understand **malware logic**.

Core concepts:

- Malware vs legitimate tool abuse
- Droppers vs loaders
- In-memory malware
- Credential theft basics
- Lateral movement basics

Key idea:

Modern malware avoids signatures — EDR detects **behavior chains**

If your mental model is still "virus.exe", stop here.

Malware Fundamentals (Behavior, Not Labels)

If your mental model is still "virus.exe", stop here.

1. Definition (Beginner-Level)

Malware fundamentals are about understanding **what malicious activity tries to achieve and how it behaves**, not what it is called.

Malware is not defined by:

- File name
- File extension
- Virus family name

Malware is defined by **actions**:

- How it executes
- How it persists
- How it steals data
- How it moves through the environment

EDR detects **behavior chains**, not labels.

2. Purpose (Why This Exists)

This topic exists to correct a dangerous mental model:

Analysts search for “malware files” instead of malicious behavior.

In modern environments:

- Malware may never exist as a file
- Legitimate tools may be abused
- The same behavior appears across many malware families

Understanding malware logic allows you to:

- Analyze alerts without relying on signatures
- Reason about intent and impact
- Detect novel or unknown threats
- Avoid being misled by benign file names

Without this shift, EDR analysis collapses into guesswork.

3. Real-World Example (Concrete)

An alert shows:

- powershell.exe executes in memory
- Contacts an external server

- Injects code into another process
- Attempts credential access

There is no malicious executable.

No hash.

No "virus.exe".

Yet the **behavior chain** clearly shows malicious activity.

4. How It Works (Internal Logic – Step by Step)

4.1 Malware vs Legitimate Tool Abuse

- **Traditional malware:**
 - Custom executables
 - Dropped to disk
- **Tool abuse:**
 - Uses built-in, trusted binaries
 - No new malware file required

Security relevance:

- Abuse produces the same outcomes
 - Detection depends on behavior, not artifacts
-

4.2 Droppers vs Loaders

- **Dropper:**
 - Initial component
 - Responsible for placing or retrieving additional code
- **Loader:**
 - Responsible for loading and executing payloads
 - Often operates in memory

Security relevance:

- Initial file may be benign-looking
- Real payload appears later or never touches disk
- EDR correlates execution stages

4.3 In-Memory Malware

- Code runs entirely in memory
- No executable written to disk
- Uses scripting engines or injection

Security relevance:

- Traditional antivirus is ineffective
- Memory behavior and process relationships are key
- Short-lived but high-impact

4.4 Credential Theft Basics

- Goal: obtain authentication material
- Targets:
 - User credentials
 - Tokens
 - Cached secrets

Security relevance:

- Often precedes lateral movement
- Direct interaction with sensitive processes
- Attempts may fail but still indicate intent

You only need to recognize **attempts**, not extraction techniques.

4.5 Lateral Movement Basics

- Goal: move from one system to another
- Uses:
 - Network protocols
 - Credentials
 - Trusted system tools

Security relevance:

- Workstations initiating admin protocols is suspicious

- Behavior depends on endpoint role
 - EDR detects unusual network + process combinations
-

5. Simplified Explanation (Non-Technical)

Think of malware like a thief:

- It may wear different clothes
- It may use your own tools
- It may never break a window

You detect the theft by **what it does**, not what it looks like.

6. Practical Scenario (Realistic Use)

Observed sequence:

- User opens a document
- Script engine runs silently
- Memory-only code executes
- Credentials are accessed
- Network connections follow

No malware file is found.

Correct analysis:

- This is a complete attack chain
 - Detection relies on behavior correlation
 - Labels are irrelevant
-

7. Common Misunderstandings

1. **“Malware must be a file”**
Incorrect. Many attacks are fileless.
2. **“If antivirus didn’t detect it, it’s fine”**
Incorrect. AV focuses on signatures.

3. **“Tool abuse is less serious than malware”**
Incorrect. Impact is identical.
 4. **“Credential theft requires success”**
Incorrect. Attempts matter.
 5. **“Lateral movement only happens on servers”**
Incorrect. Workstations are common entry points.
-

8. Summary (Concise)

- Malware is defined by behavior, not names
 - Legitimate tools are commonly abused
 - Droppers initiate, loaders execute
 - In-memory malware avoids disk artifacts
 - Credential theft signals escalation intent
 - Lateral movement expands impact
 - EDR detects behavior chains, not signatures
 - “virus.exe” thinking is obsolete
-

9. Identity & Credential Abuse on Endpoints

EDR tracks **identity misuse**, not just processes.

You must understand:

- How credentials are used locally
- Why LSASS access is dangerous
- Token impersonation (high-level)
- Pass-the-hash (conceptual)

Alerts involving LSASS or token access are **high-risk**.
If you don't know why, context is lost.

Identity & Credential Abuse on Endpoints

EDR tracks identity misuse, not just processes.

1. Definition (Beginner-Level)

Identity and credential abuse on endpoints refers to **misuse of authentication material and security tokens on a local machine**, rather than simply running malicious programs.

On Windows:

- Every process runs under an identity
- Credentials and tokens determine **what a process is allowed to do**
- Attacks often focus on **stealing or misusing identity**, not dropping files

EDR monitors **how identities are accessed and used**, not just which processes run.

2. Purpose (Why This Exists)

This topic exists to correct a narrow detection mindset:

Analysts focus on suspicious processes and miss identity abuse that enables everything else.

In real environments:

- Credential access enables persistence, lateral movement, and escalation
- Identity misuse may occur **without obvious malware**
- Some of the highest-impact attacks start with credential access

Understanding identity abuse allows you to:

- Correctly prioritize high-risk alerts
- Recognize escalation attempts early
- Understand why certain alerts are critical even if execution looks benign

Alerts involving LSASS or token access are high-risk **by definition**, not by verdict.

3. Real-World Example (Concrete)

An EDR alert shows:

- A normal-looking process
- Attempting to read memory associated with authentication
- No new process creation
- No malware file

If you only think in terms of processes:

- You may dismiss this as noise

If you understand identity abuse:

- You recognize this as an attempt to access credentials
 - You treat it as a potential escalation point
-

4. How It Works (Internal Logic – Step by Step)

4.1 How Credentials Are Used Locally

- When a user logs in, Windows validates credentials
- Authentication material is used to create a **security token**
- Processes inherit tokens from their parent

Security relevance:

- Stealing credentials allows impersonation
 - Tokens define access, not the user interface
 - Abuse can occur silently in the background
-

4.2 Why LSASS Access Is Dangerous

- LSASS manages authentication and credential material
- Credentials are handled in memory

Security relevance:

- Accessing LSASS memory is rarely legitimate
- Attempts indicate credential theft intent
- Even failed access attempts are meaningful

EDR treats LSASS interaction as **high-sensitivity behavior**.

4.3 Token Impersonation (High-Level)

- Tokens represent identity and privileges
- A process can attempt to act as another user by using a token

Security relevance:

- Allows privilege escalation
- Allows access without knowing the password
- Often used after initial compromise

Understanding the concept is sufficient — no implementation detail is required.

4.4 Pass-the-Hash (Conceptual)

- Authentication can rely on credential representations, not plaintext passwords
- Stolen material can be reused without cracking it

Security relevance:

- Enables lateral movement
- Bypasses normal login behavior
- Often paired with credential access attempts

EDR detects **use patterns**, not the cryptographic details.

5. Simplified Explanation (Non-Technical)

Think of identity like a security badge:

- The badge lets you open doors
- You don't need the person — just the badge
- Stealing or copying the badge is more powerful than breaking a door

Security focuses on **badge misuse**, not just door-breaking.

6. Practical Scenario (Realistic Use)

You receive an alert:

- A background process attempts to access LSASS memory
- The process is signed
- No obvious malicious execution occurred

Correct reasoning:

- Signed does not mean safe
- LSASS access is not normal behavior
- This indicates credential access intent

Action:

- Treat as high-risk
- Correlate with user activity and network behavior
- Do not dismiss due to lack of malware file

7. Common Misunderstandings

1. **"No malware file means no credential theft"**
Incorrect. Identity abuse is often fileless.
2. **"Only admin accounts are targeted"**
Incorrect. Any credential can be leveraged.
3. **"If access failed, it's harmless"**
Incorrect. Attempted access indicates intent.
4. **"Token abuse requires passwords"**
Incorrect. Tokens bypass passwords.
5. **"Identity alerts are secondary to execution alerts"**
Incorrect. Identity abuse enables everything else.

8. Summary (Concise)

- Identity misuse is central to endpoint attacks
 - Credentials create tokens that define access
 - LSASS manages sensitive authentication material
 - LSASS access attempts are inherently high-risk
 - Token impersonation enables escalation
 - Pass-the-hash reuses credential material
 - EDR tracks identity behavior, not just processes
 - Loss of identity context leads to misanalysis
-

10. Attack Lifecycle & Tradecraft Thinking

EDR maps attacker behavior across stages.

You must understand the **logic**, not framework names:

- Initial access
- Execution
- Persistence
- Privilege escalation
- Defense evasion
- Command & control

Also understand attacker tradecraft:

- Why attackers blend into normal tools
- Why low-and-slow beats noisy attacks
- Why admin privileges are avoided early

EDR only makes sense when you think like the attacker.

Attack Lifecycle & Tradecraft Thinking

EDR maps attacker behavior across stages — it only makes sense when you think like the attacker.

1. Definition (Beginner-Level)

Attack lifecycle thinking means understanding an attack as a **sequence of logical stages**, not isolated alerts.

An attacker does not:

- Run one command
- Trigger one alert

- Finish the attack

Instead, attackers move through **phases**, each enabling the next.

EDR exists to **observe, correlate, and connect these phases over time**.

2. Purpose (Why This Exists)

This topic exists to solve a core analytical failure:

Analysts treat alerts as independent events instead of parts of a campaign.

In real environments:

- Each stage may look low-risk on its own
- The combination reveals intent
- EDR's value comes from **correlation across time and behavior**

Understanding the attack lifecycle allows you to:

- Identify where the attacker currently is
- Predict what comes next
- Prioritize response actions
- Avoid alert-by-alert tunnel vision

Without lifecycle thinking, EDR output is fragmented and misleading.

3. Real-World Example (Concrete)

Observed over several hours:

- A user opens a document
- A script runs silently
- A registry key is added
- A network beacon occurs
- Credentials are accessed later

Each event alone might look benign.

Together, they form a **complete attack chain**.

EDR connects these behaviors across stages.

4. How It Works (Internal Logic – Step by Step)

4.1 Initial Access

- First foothold on the system
- Often low-noise and user-driven

Security relevance:

- Looks like normal user activity
 - Often ignored or missed
 - Sets the stage for everything else
-

4.2 Execution

- Code is run on the endpoint
- Often uses trusted binaries or scripts

Security relevance:

- Execution may be brief
 - No malware file required
 - Context matters more than content
-

4.3 Persistence

- Mechanisms added to re-run code
- Survives reboot or logoff

Security relevance:

- Strong indicator of intent
 - Often higher confidence than execution
 - Long-lived visibility for defenders
-

4.4 Privilege Escalation

- Attempts to gain higher privileges
- Not always immediate

Security relevance:

- Early escalation is noisy
 - Failed attempts still matter
 - Token and credential activity is key
-

4.5 Defense Evasion

- Attempts to avoid detection or blocking
- Can happen at any stage

Security relevance:

- Includes blending in with normal tools
 - Includes disabling or bypassing protections
 - Often subtle and indirect
-

4.6 Command & Control

- Communication with external systems
- Enables attacker control and updates

Security relevance:

- Often low-volume and periodic
 - Uses common protocols
 - May appear harmless without context
-

5. Simplified Explanation (Non-Technical)

Think of an attacker like a burglar:

- First enters quietly

- Moves carefully
- Avoids alarms
- Sets up a way to return
- Communicates with helpers

Security catches burglars by **watching patterns**, not single actions.

6. Practical Scenario (Realistic Use)

You see:

- A low-severity execution alert
- Later, a persistence alert
- Later, credential access

Correct interpretation:

- These are not separate incidents
- They represent progression
- The attacker is advancing through stages

Response should be based on **where the attacker is**, not alert count.

7. Common Misunderstandings

1. **"Each alert is its own incident"**
Incorrect. Alerts are fragments of a lifecycle.
 2. **"Early-stage activity is low risk"**
Incorrect. Early stages enable everything else.
 3. **"Attackers escalate privileges immediately"**
Incorrect. Early escalation increases detection risk.
 4. **"Defense evasion only happens late"**
Incorrect. It occurs throughout the attack.
 5. **"Framework names equal understanding"**
Incorrect. Logic matters, labels do not.
-

8. Attacker Tradecraft (Explicit Reasoning)

Why Attackers Blend into Normal Tools

- Trusted binaries reduce suspicion
 - Allowed by policy
 - Fewer alerts triggered
-

Why Low-and-Slow Beats Noisy Attacks

- Reduces detection probability
 - Blends with background activity
 - Exploits alert fatigue
-

Why Admin Privileges Are Avoided Early

- Escalation is noisy
- High-risk operations draw attention
- Attackers wait until position is stable

These choices are **strategic**, not technical.

9. Summary (Concise)

- Attacks progress through logical stages
 - EDR correlates behavior across time
 - Individual alerts lack meaning alone
 - Lifecycle thinking reveals intent
 - Persistence and identity abuse are high-signal
 - Tradecraft favors stealth over speed
 - Admin access is delayed intentionally
 - EDR analysis requires attacker mindset
-

11. Logging, Telemetry & Correlation Thinking

EDR is **high-fidelity telemetry**, not alert spam.

You must understand:

- What an event is
- What metadata is
- Noise vs signal
- Why false positives happen
- Time-based and cross-process correlation

One alert $\neq$ one incident.

Investigations are **graphs**, not lists.

Logging, Telemetry & Correlation Thinking

EDR is high-fidelity telemetry, not alert spam.

1. Definition (Beginner-Level)

Logging and telemetry are records of **what actually happened on an endpoint**, captured as discrete data points over time.

- An **event** is a single recorded action (for example: a process start).
- **Telemetry** is the continuous stream of such events.
- **Correlation** is the act of connecting related events to understand behavior.

EDR collects telemetry first.

Alerts are a **derived interpretation**, not the raw truth.

2. Purpose (Why This Exists)

This topic exists to correct a destructive habit:

Treating alerts as facts instead of treating telemetry as evidence.

In real environments:

- Thousands of events occur per minute
- Alerts are summaries, not full context
- Single alerts rarely represent full attacks

Understanding telemetry and correlation allows you to:

- Reconstruct what actually happened
- Separate benign activity from malicious chains
- Avoid alert-by-alert tunnel vision
- Perform investigations instead of verdict acceptance

One alert ≠ one incident.

3. Real-World Example (Concrete)

You see an alert:

- "Suspicious PowerShell execution"

Telemetry reveals:

- Process creation event
- Command-line metadata
- Parent process event
- Network connection event minutes later
- Registry modification event after that

The alert is a **headline**.

The telemetry is the **story**.

4. How It Works (Internal Logic – Step by Step)

4.1 What an Event Is

- A single recorded action at a point in time
- Examples:
 - Process creation
 - File write
 - Registry change
 - Network connection

Security relevance:

- Events are atomic
 - They are factual
 - They are meaningless alone without context
-

4.2 What Metadata Is

- Descriptive information attached to events
- Examples:
 - Timestamp
 - Process ID
 - Parent process
 - User identity
 - File path
 - Command-line arguments

Security relevance:

- Metadata provides **context**
 - Most detections rely on metadata, not the event itself
 - Missing or ignored metadata leads to misinterpretation
-

4.3 Noise vs Signal

- **Noise:**
 - Expected background activity
 - Repetitive, low-risk behavior
- **Signal:**
 - Behavior that deviates from normal patterns
 - Contextually abnormal sequences

Security relevance:

- Noise is unavoidable
 - Signal emerges through comparison and correlation
 - Analysts must learn to ignore volume and focus on relevance
-

4.4 Why False Positives Happen

False positives occur because:

- Legitimate tools perform powerful actions
- Context is incomplete or misinterpreted
- Generic rules cannot account for every environment

Security relevance:

- A false positive is not “random”
 - It is a **misclassified behavior**
 - Understanding why it fired improves detection quality
-

4.5 Time-Based Correlation

- Events gain meaning when ordered in time
- Earlier actions explain later ones

Security relevance:

- Initial access may occur long before impact
- Delayed actions are common
- Time gaps do not imply unrelated activity

EDR links events across minutes, hours, or days.

4.6 Cross-Process Correlation

- One process triggers another
- Chains span multiple executables

Security relevance:

- Attacks rarely stay in one process
- Parent–child relationships matter
- Cross-process chains reveal intent

EDR tracks **flows**, not isolated executables.

5. Simplified Explanation (Non-Technical)

Think of telemetry like CCTV footage:

- Each camera frame is an event
- Metadata is the timestamp and camera ID
- One frame proves nothing
- A sequence shows what actually happened

Investigations require **replaying the footage**, not reading headlines.

6. Practical Scenario (Realistic Use)

You investigate:

- Three low-severity alerts
- Spread across several hours

Telemetry shows:

- Same user
- Same endpoint
- Same execution chain
- Progressive escalation of actions

Correct conclusion:

- This is one incident
 - The alerts are fragments
 - Correlation reveals intent and progression
-

7. Common Misunderstandings

1. **"Alerts are the evidence"**
Incorrect. Telemetry is the evidence.
 2. **"High alert volume means compromise"**
Incorrect. Volume often equals noise.
 3. **"False positives are useless"**
Incorrect. They reveal detection boundaries.
 4. **"Time gaps break attack chains"**
Incorrect. Attackers wait intentionally.
 5. **"One alert equals one attacker action"**
Incorrect. Alerts summarize multiple events.
-

8. Summary (Concise)

- EDR collects high-fidelity telemetry
 - Events are atomic facts
 - Metadata provides essential context
 - Noise is expected; signal is contextual
 - False positives arise from generic logic
 - Time-based correlation reveals progression
 - Cross-process correlation reveals chains
 - One alert is never the full story
 - Investigations are graphs, not lists
-

12. Baselines & Environment Context

Behavior is **relative**, not absolute.

You must understand:

- Developer vs user endpoints
- Admin workstations vs kiosks
- Why PowerShell is normal in some environments

EDR without context creates false confidence.

Baselines & Environment Context

Behavior is relative, not absolute.

1. Definition (Beginner-Level)

A **baseline** is an understanding of **what normal behavior looks like in a specific environment**.

Environment context means:

- Who uses the endpoint
- What the endpoint is used for
- What tools and behaviors are expected there

There is no universal "normal."

The same action can be benign on one endpoint and suspicious on another.

EDR evaluates behavior **relative to context**, not in isolation.

2. Purpose (Why This Exists)

This topic exists to prevent a critical analytical failure:

Treating all endpoints as identical and all behavior as absolute.

In real environments:

- Different roles require different tools
- The same binary can be routine or dangerous
- Alerts without context lead to false confidence or false panic

Understanding baselines allows you to:

- Reduce false positives
- Correctly prioritize alerts
- Avoid mislabeling legitimate activity as malicious

- Detect real abuse hidden inside “normal” behavior

EDR without context does not increase security — it creates false confidence.

3. Real-World Example (Concrete)

Two endpoints show:

- Frequent powershell.exe execution
- Script-based automation
- Network activity following execution

Endpoint A:

- Used by a developer
- Automation scripts are part of daily work

Endpoint B:

- Used as a front-desk kiosk
- No scripting should occur

Same behavior.

Different conclusions.

Only context determines whether this is normal or suspicious.

4. How It Works (Internal Logic – Step by Step)

4.1 Developer vs User Endpoints

- **Developer endpoints:**
 - Frequent scripting
 - Compilation tools
 - Debuggers
 - Automation frameworks
- **User endpoints:**
 - Office applications
 - Browsers
 - Minimal scripting

Security relevance:

- High-risk binaries may be normal for developers
 - Blocking them blindly disrupts work
 - Detection must focus on *deviations from that baseline*
-

4.2 Admin Workstations vs Kiosks

- **Admin workstations:**
 - Elevated privileges
 - Remote management tools
 - System-level access
- **Kiosks:**
 - Locked-down usage
 - Single-purpose
 - Minimal interaction

Security relevance:

- Admin activity on kiosks is suspicious
 - Kiosk-like behavior on admin machines is irrelevant
 - Endpoint role defines acceptable risk
-

4.3 Why PowerShell Is Normal in Some Environments

- PowerShell is:
 - A legitimate administration tool
 - A scripting and automation engine
- Some environments rely on it heavily

Security relevance:

- PowerShell presence is not the signal
- Context, arguments, and execution chains are
- Over-alerting on normal admin behavior hides real threats

Understanding usage patterns matters more than binary names.

5. Simplified Explanation (Non-Technical)

Think of behavior like clothing:

- A uniform is normal at work
- The same outfit is strange at a wedding

Security judgments depend on **where and why**, not just **what**.

6. Practical Scenario (Realistic Use)

An EDR alert flags:

- PowerShell execution
- Network connections
- Script execution

Analysis steps:

- Identify endpoint role
- Identify user role
- Compare behavior to historical baseline
- Determine deviation, not presence

Correct outcome:

- Developer machine → likely normal
- Shared kiosk → high-risk

Same alert.

Different response.

7. Common Misunderstandings

1. **“PowerShell is always suspicious”**
Incorrect. Context defines legitimacy.
2. **“One baseline fits all endpoints”**
Incorrect. Roles differ.
3. **“Admins should never trigger alerts”**
Incorrect. Admin abuse is high-impact.

4. **"False positives mean bad detection"**
Incorrect. They often mean missing context.
 5. **"EDR severity equals importance"**
Incorrect. Context overrides generic scoring.
-

8. Summary (Concise)

- Behavior is relative to environment
 - Baselines define what "normal" means
 - Endpoint role matters
 - User role matters
 - Same behavior can be benign or malicious
 - PowerShell is normal in some environments
 - Context prevents false confidence
 - EDR without baselines misleads analysis
-

13. Incident Response & Response Impact

Before touching EDR response actions, understand:

- Containment vs eradication
- Evidence preservation
- Business and availability impact
- Host isolation consequences
- Killing processes vs breaking applications

EDR response must be **surgical**, not emotional.

Incident Response & Response Impact

EDR response must be surgical, not emotional.

1. Definition (Beginner-Level)

Incident response is the set of actions taken **after suspicious or malicious activity is identified** to limit damage, preserve evidence, and restore normal operations.

EDR response actions are **not neutral**:

- Every action changes system state
- Some actions destroy evidence
- Some actions disrupt business operations

Response is part of the incident, not separate from it.

2. Purpose (Why This Exists)

This topic exists to prevent a common operational failure:

Analysts act immediately on alerts without understanding consequences.

In real environments:

- Overreaction causes outages
- Premature cleanup destroys evidence
- "Fixing" the system can help the attacker by hiding traces

Understanding response impact allows you to:

- Choose the correct action at the correct time
- Balance security with availability
- Preserve forensic value
- Avoid turning detection into self-inflicted damage

EDR response must be deliberate and informed, not reflexive.

3. Real-World Example (Concrete)

An alert indicates:

- Suspicious process activity
- Possible credential access

An analyst:

- Immediately kills the process
- Isolates the host
- Deletes related files

Result:

- The attacker's method is unknown
- Evidence is lost
- A critical business application is down

The alert was real — the response was wrong.

4. How It Works (Internal Logic – Step by Step)

4.1 Containment vs Eradication

- **Containment:**
 - Limits spread
 - Stops further damage
 - Temporary by design
- **Eradication:**
 - Removes attacker presence
 - Requires understanding of scope

Security relevance:

- Containment comes first
- Eradication without understanding causes reinfection or data loss
- EDR actions should support containment before cleanup

4.2 Evidence Preservation

- Telemetry, logs, memory, and artifacts are evidence

- Some actions permanently destroy data

Security relevance:

- Killing processes removes memory-resident evidence
- Deleting files removes forensic artifacts
- Isolation may cut off command-and-control visibility

Evidence should be preserved **before** destructive actions.

4.3 Business and Availability Impact

- Endpoints support real work
- Security actions can stop that work

Security relevance:

- Not all endpoints are equal
- Taking down a kiosk is different from taking down a production system
- Response must consider business criticality

EDR response is a risk decision, not just a security decision.

4.4 Host Isolation Consequences

- Isolation cuts network connectivity
- Prevents lateral movement and data exfiltration

Security relevance:

- Isolation may:
 - Break remote access
 - Stop monitoring
 - Alert the attacker

Isolation is powerful but disruptive.
It should be used intentionally, not automatically.

4.5 Killing Processes vs Breaking Applications

- Processes may be shared by applications
- Killing one process can crash dependent services

Security relevance:

- Terminating a process may:
 - Remove attacker access
 - Destroy memory evidence
 - Break legitimate applications

Understanding process role is required before termination.

5. Simplified Explanation (Non-Technical)

Incident response is like surgery:

- You don't cut everything
- You don't rush without diagnosis
- Every action has side effects

Good response removes the threat **without killing the patient**.

6. Practical Scenario (Realistic Use)

You detect:

- Suspicious PowerShell activity
- No confirmed persistence
- No lateral movement yet

Correct approach:

- Monitor additional behavior
- Preserve telemetry
- Consider containment options
- Delay destructive actions until scope is understood

Wrong approach:

- Immediately isolate and kill processes without context

EDR response should slow the attacker, not blind the defender.

7. Common Misunderstandings

1. **"Fast response is always better"**
Incorrect. Correct response is better.
 2. **"Isolation is harmless"**
Incorrect. It disrupts operations and visibility.
 3. **"Killing a process removes the threat"**
Incorrect. It may remove evidence and miss persistence.
 4. **"EDR response equals incident closure"**
Incorrect. Response is one phase of the incident.
 5. **"More actions mean stronger security"**
Incorrect. Unnecessary actions increase risk.
-

8. Summary (Concise)

- Incident response actions change system state
 - Containment and eradication are different goals
 - Evidence must be preserved before cleanup
 - Business impact matters
 - Host isolation is disruptive but effective
 - Killing processes can break applications and destroy evidence
 - EDR response must be precise and intentional
 - Emotional reactions create damage
-

14. Analyst Discipline (Mandatory)

Before touching an EDR console:

- Never trust alerts blindly
- Always validate context
- Document assumptions
- Think in hypotheses, not conclusions

EDR amplifies skill — **good or bad.**

Analyst Discipline (Mandatory)

EDR amplifies skill — good or bad.

1. Definition (Beginner-Level)

Analyst discipline is the set of **mental rules and habits** an analyst must follow **before and during** any interaction with an EDR console.

It is not a tool skill.

It is a **thinking discipline**.

EDR does not make decisions.

Analysts do.

2. Purpose (Why This Exists)

This topic exists to address the most dangerous failure mode in security operations:

Treating EDR output as truth instead of evidence.

In real environments:

- Alerts are imperfect
- Context is incomplete
- Automated verdicts are probabilistic

Analyst discipline ensures that:

- Errors are caught early
- Assumptions are made explicit
- Decisions are defensible
- Skill gaps are not hidden behind tooling

EDR amplifies analyst thinking — correct or incorrect.

3. Real-World Example (Concrete)

An alert states:

- “High-confidence malware detected”

An undisciplined analyst:

- Accepts the verdict
- Isolates the host
- Closes the incident

A disciplined analyst:

- Reviews telemetry
- Validates execution context
- Questions why the alert fired
- Confirms whether behavior is actually malicious

Same alert.

Different outcomes.

4. How It Works (Internal Logic – Step by Step)

4.1 Never Trust Alerts Blindly

- Alerts are interpretations

- They are generated by rules and models

Security relevance:

- Rules lack full context
- Models generalize across environments
- Blind trust leads to false confidence

Alerts must be **validated**, not accepted.

4.2 Always Validate Context

Context includes:

- Endpoint role
- User role
- Baseline behavior
- Execution chain
- Timing and sequence

Security relevance:

- Context determines meaning
- Same behavior can be benign or malicious
- Validation prevents knee-jerk response

Without context, alerts are ambiguous.

4.3 Document Assumptions

- Every investigation includes assumptions
- Undocumented assumptions become hidden bias

Security relevance:

- Assumptions influence conclusions
- Documentation allows review and correction
- Clear assumptions improve handoffs and audits

An undocumented assumption is a silent risk.

4.4 Think in Hypotheses, Not Conclusions

- A **hypothesis** is testable
- A **conclusion** is final

Security relevance:

- Early conclusions cause tunnel vision
- Hypotheses encourage evidence gathering
- Multiple hypotheses reduce confirmation bias

Good analysts ask:

- "What could explain this?"
Not:
- "This must be malware."

5. Simplified Explanation (Non-Technical)

Think like a detective:

- You don't arrest on rumors
- You gather evidence
- You test theories
- You write down what you assume

EDR gives clues — not answers.

6. Practical Scenario (Realistic Use)

You see:

- A suspicious PowerShell alert

Disciplined approach:

- Hypothesis A: malicious execution
- Hypothesis B: admin automation

- Hypothesis C: false positive

You test each using:

- Telemetry
- Context
- Baseline behavior

Only then do you decide on response.

7. Common Misunderstandings

1. **"High severity means high confidence"**
Incorrect. Severity $\neq$ truth.
 2. **"The tool already analyzed it"**
Incorrect. The tool summarized it.
 3. **"Documenting assumptions slows response"**
Incorrect. It prevents mistakes.
 4. **"Experienced analysts can skip validation"**
Incorrect. Experience increases bias risk.
 5. **"Confidence equals correctness"**
Incorrect. Evidence equals correctness.
-

8. Summary (Concise)

- Analyst discipline is mandatory, not optional
 - Alerts are interpretations, not facts
 - Context validation is required
 - Assumptions must be documented
 - Hypothesis-driven thinking prevents tunnel vision
 - EDR amplifies analyst skill and bias
 - Poor discipline turns EDR into a liability
-

FINAL SUMMARY

- EDR is a telemetry and correlation system, not a malware finder
- EDR amplifies analyst skill and bias; it does not replace thinking
- Understanding Windows endpoint internals is mandatory to interpret alerts
- Modern attacks are process-, memory-, and behavior-centric, not file-centric
- Signed and trusted binaries are commonly abused; absence of malware files is normal
- EDR observes runtime behavior and API activity, not attacker intent
- Windows security controls (identity, tokens, LSASS, permissions, logs) exist before EDR and provide the foundation for detections
- Persistence mechanisms and credential access are high-confidence indicators of malicious intent
- Identity abuse enables escalation, lateral movement, and long-term access
- Network detections rely on endpoint context, not packet inspection
- Attacks unfold as lifecycles; alerts are fragments, not standalone incidents
- Telemetry and metadata are evidence; alerts are summaries
- Effective investigations require time-based and cross-process correlation
- Behavior must be evaluated against environment baselines and endpoint roles
- Response actions change system state and can destroy evidence or disrupt business
- Containment, eradication, and isolation must be deliberate and minimal
- Analysts must validate context, document assumptions, and think in hypotheses
- Without discipline and foundational knowledge, EDR produces noise and false confidence